

Computer Organization, Spring 2026

Project Assignment 1: Multiplier and Divider Design

Lecturer: 陳雅淑教授

Department: Electrical Engineering

Student ID: B11230024

Name: 王映媛

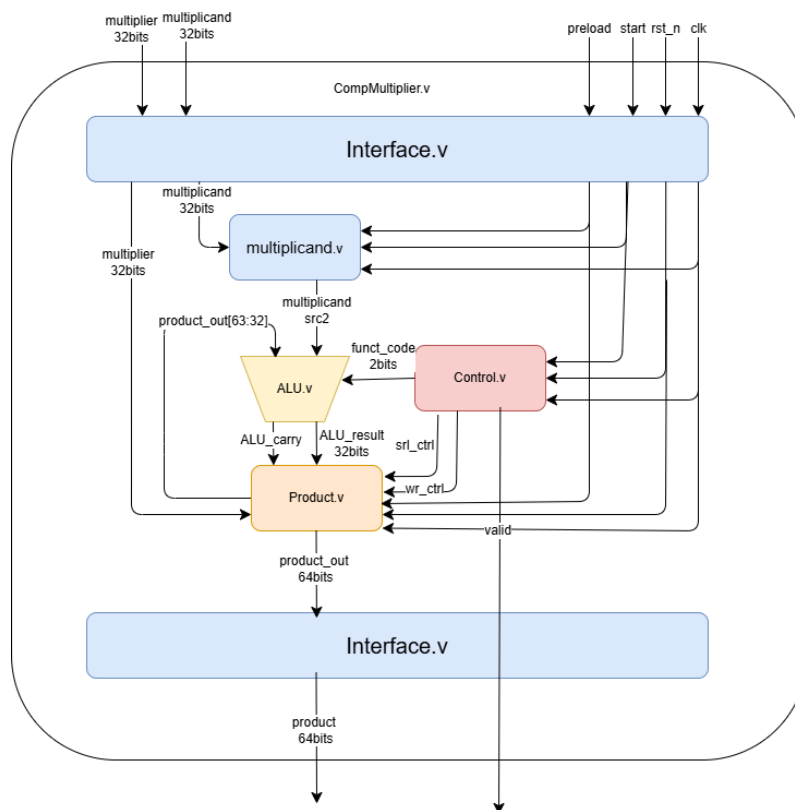
1. Describe how you implement each module

Part1. Multiplier

(a) CompMultiplier.v

```
module CompMultiplier (
    input clk, rst_n, preload, start,
    input [31:0] multiplier, multiplicand,
    output [63:0] product,
    output valid
);
    wire start_w, preload_w;
    wire [31:0] multiplier_w, multiplicand_w, multiplicand_out_w, alu_result_w;
    wire alu_carry_w, srl_ctrl_w, wr_ctrl_w, valid_w;
    wire [1:0] funct_code_w;
    wire [63:0] product_w;
    Interface M1 (
        .clk(clk), .rst_n(rst_n), .start(start), .preload(preload),
        .multiplier(multiplier), .multiplicand(multiplicand), .product_in(product_w), .valid(valid_w),
        .start_out(start_w), .preload_out(preload_w),
        .multiplier_out(multiplier_w), .multiplicand_out(multiplicand_w), .product(product_w),
        .valid_in(valid_w)
    );
    Multiplicand M2 (
        .clk(clk), .rst_n(rst_n), .preload(preload_w),
        .multiplicand(multiplicand_w), .multiplicand_out(multiplicand_out_w)
    );
    Product M3 (
        .clk(clk), .rst_n(rst_n), .preload(preload_w), .wr_ctrl(wr_ctrl_w), .srl_ctrl(srl_ctrl_w),
        .multiplier(multiplier_w), .ALU_result(alu_result_w), .ALU_carry(alu_carry_w),
        .product_out(product_w)
    );
    ALU M4(
        .src_1(product_w[63:32]), .src_2(multiplicand_out_w),
        .funct_code(funct_code_w), .ALU_result(alu_result_w), .ALU_carry(alu_carry_w)
    );
    Control M5(
        .clk(clk), .rst_n(rst_n),
        .start(start_w),
        .LSB(product_w[0]),
        .funct_code(funct_code_w),
        .srl_ctrl(srl_ctrl_w),
        .wr_ctrl(wr_ctrl_w),
        .valid(valid_w)
    );
endmodule
```

圖一、CompMultiplier.v



圖二、Multiplier 架構圖

CompMultiplier.v 將 Multiplicand,ALU,Product,Control 各模組線路照架構圖(圖二)內容做連接。

(b.) Interface.v

```
module Interface(  
    input clk,rst_n,start,preload,  
    input [31:0] multiplier,multiplicand,  
    input [63:0] product_in,  
    input valid_in,  
    output reg start_out,preload_out,  
    output reg [31:0] multiplier_out,multiplicand_out,  
    output reg [63:0] product,  
    output reg valid  
);  
always@(posedge clk)  
begin  
    if (!rst_n)  
    begin  
        start_out<=1'b0;  
        preload_out<=1'b0;  
        multiplier_out<=32'b0;  
        multiplicand_out<=32'b0;  
        product<=64'b0;  
        valid<=1'b0;  
    end  
    else  
    begin  
        preload_out<=preload;  
        start_out<=start;  
        multiplier_out<=multiplier;  
        multiplicand_out<=multiplicand;  
        product<=product_in;  
        valid<=valid_in;  
    end  
end  
endmodule
```

圖三、Interface.v

Interface.v 負責外部測試與內部運算間的訊號同步與隔離。電路採時脈正緣觸發、同步低準位重置，確保系統重置時所有輸出端清零。在正常運作階段，Interface.v 將外部輸入的 start、preload、multiplier、multiplicand、product_in、valid_in 透過暫存器鎖存一個時鐘週期後同步傳遞至輸出端。此設計有效對齊所有訊號的時序，避免外部延遲或雜訊影響內部電路的穩定性。

(c) Multiplicand.v

```
module Multiplicand(  
    input clk,rst_n,preload,  
    input [31:0] multiplicand,  
    output reg [31:0] multiplicand_out  
);  
always@(posedge clk)  
begin  
    if (!rst_n) multiplicand_out<=32'b0;  
    else if (preload) multiplicand_out<=multiplicand;  
end  
endmodule
```

圖四、Multiplicand.v

Multiplicand.v 為一個由 32 位元同步暫存器。正緣觸發 (posedge clk)、具備同步低準位重置 (rst_n) 功能以初始化。運算初期，透過訊號 preload 為高準位來寫入被乘數；載入後 preload 降為低準位，暫存器保持該數值不變，確保在整個乘法疊代過程中，能持續穩定提供被乘數給 ALU 進行運算。

(d)ALU.v

```
module ALU(  
    input [31:0] src_1,src_2,  
    input [1:0] funct_code,  
    output reg [31:0] ALU_result,  
    output reg ALU_carry  
);  
always@(*)  
begin  
    case(funct_code)  
        2'b01: {ALU_carry,ALU_result}=src_1+src_2;  
        default:  
            begin  
                ALU_result=src_1;  
                ALU_carry=1'b0;  
            end  
    endcase  
end  
endmodule
```

圖五、ALU.v

ALU.v 負責 Control.v 判斷目前需要進行 ADD 時，將 Product[63:32]+Multiplicand，其他時候維持 Product[63:32]。值得注意的是後面 Product.v 向右位移時，須將 Carry 引入右移。此外，funct_code 其實可以指定為 2bit，因為題目只需做加法 2bit 即可完成功能。

(e)Product.v

```
module Product(  
    input clk, rst_n, preload, wr_ctrl, srl_ctrl,  
    input [31:0] multiplier,  
    input [31:0] ALU_result,  
    input ALU_carry,  
    output reg [63:0] product_out  
);  
    reg carry; //避免ALU_carry右移時被覆蓋  
    always@(posedge clk)  
    begin  
        if (!rst_n)  
        begin  
            product_out<=64'b0;  
            carry<=1'b0;  
        end  
        else if (preload)  
        begin  
            product_out<={32'b0,multiplier};  
            carry<=1'b0;  
        end  
        else if (wr_ctrl)  
        begin  
            product_out[63:32]<=ALU_result;  
            carry<=ALU_carry;  
        end  
        else if (srl_ctrl)  
        begin  
            product_out<={carry,product_out[63:1]};  
            carry<=1'b0;  
        end  
    end  
endmodule
```

圖六、Product.v

Product.v 為一個 64 位元移位暫存器，負責在乘法運算中動態儲存與更新部分積。額外設計一個 carry 暫存器，用以存 ALU 的進位值 ALU_carry，防止移位過程中遺失資料。

當 preload 訊號觸發時，模組會將高 32 位元清零並於低 32 位元載入乘數 (Multiplier) 進行初始化。運算期間，若 wr_ctrl 高位，將 ALU 的運算結果更新至暫存器的高 32 位元；當移位控制訊號 srl_ctrl 觸發時，執行向右移位，並同步將暫存的 carry 值補入最高位元，透過 32 次的加法、移位循環，產出完整的 64 位元乘法結果。

(f)Control.v

Control.v 為有限狀態機 (FSM)，負責協調乘法器的運算時序與控制訊號。狀態機分為四個狀態：IDLE (重置)、ADD (判斷加法)、SHIFT (移位與計數) 及 DONE (完成)。在 IDLE 狀態下重置計數器 counter，隨後進入 ADD 與 SHIFT 的循環。當狀態處於 ADD 時，根據乘數的最低位元 LSB 來決定是否加法；若 LSB 為高準位，則發出寫入控制 wr_ctrl 並 ALU 執行加法。每完成一次加法判斷後，狀態機轉入 SHIFT 狀態發移位訊號 srl_ctrl，並累加計數器。當循環執行達 32 次，狀態機轉入 DONE 輸出完成訊號 valid，隨後回到閒置狀態等待下一次運算開始。

Part2. Divider

(a) CompDivider.v

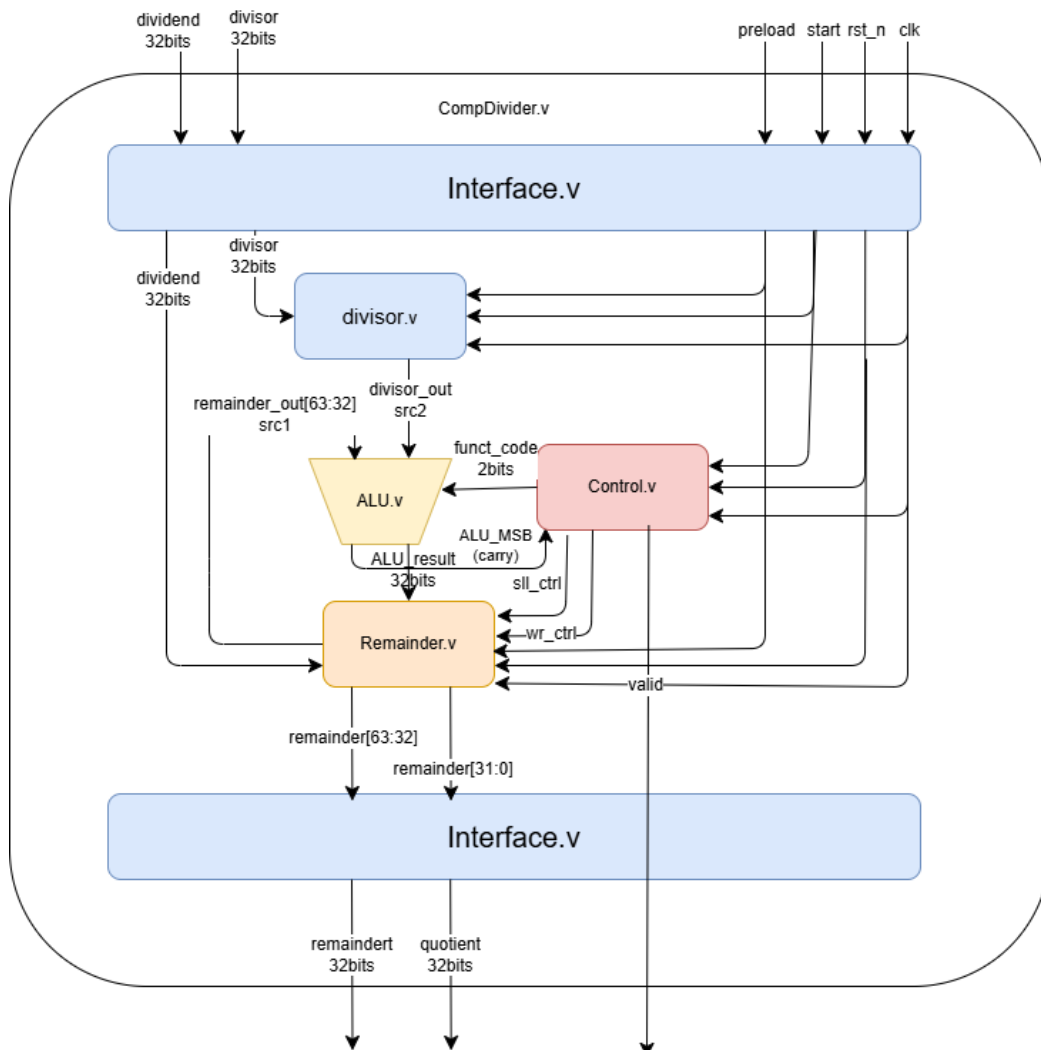
CompDivider.v 將 Divisor,ALU,Remainder,Control 各模組線路照架構圖(圖八)內容做連接。

```

module CompDivider (
    input      clk,rst_n,preload,start,
    input  [31:0] divisor,dividend,
    output [31:0] remainder,quotient,
    output      valid
);
    wire      start_w, preload_w;
    wire [31:0] divisor_w, dividend_w,divisor_out_w,alu_result_w;
    wire      alu_carry_w,sll_ctrl_w, wr_ctrl_w, valid_w;
    wire [1:0]funcnt_code_w;
    wire [63:0] remainder_w;
    wire [31:0] quotient_w;
    Interface M1 (
        .clk(clk),.rst_n(rst_n),.start(start),.preload(preload),.dividend(dividend),
        .divisor(divisor),.remainder_in(remainder_w),.quotient_in (remainder_w[31:0]),
        .valid_in(valid_w),.start_out(start_w),.preload_out(preload_w),.dividend_out(dividend_w),
        .divisor_out(divisor_w),.remainder(remainder),.quotient(quotient),.valid(valid)
    );
    Divisor M2 (
        .clk(clk),.rst_n(rst_n),.preload(preload_w),.divisor(divisor_w),.divisor_out(divisor_out_w)
    );
    Remainder M3 (
        .clk(clk),.rst_n(rst_n),.preload(preload_w),.wr_ctrl(wr_ctrl_w),.sll_ctrl(sll_ctrl_w),
        .dividend(dividend_w),.ALU_result(alu_result_w),.remainder_out(remainder_w)
    );
    ALU u_ALU (
        .src_1(remainder_w[63:32]),.src_2(divisor_out_w),.funcnt_code(funcnt_code_w),
        .ALU_carry (alu_carry_w),.ALU_result(alu_result_w)
    );
    Control u_Control (
        .clk(clk),.rst_n(rst_n),.start(start_w),.ALU_MSB(alu_carry_w),.funcnt_code(funcnt_code_w),
        .sll_ctrl(sll_ctrl_w),.wr_ctrl(wr_ctrl_w),.valid(valid_w)
    );
endmodule

```

圖七、CompDivider



圖八、Divider 架構圖

(b)Interface.v

```
module Interface (  
    input clk, rst_n, start, preload,  
    input [31:0] dividend, divisor,  
    input [63:0] remainder_in,  
    input [31:0] quotient_in,  
    input valid_in,  
    output reg start_out, preload_out,  
    output reg [31:0] dividend_out, divisor_out, remainder, quotient,  
    output reg valid  
);  
always @(posedge clk) begin  
    if (!rst_n) begin  
        start_out    <= 1'b0;  
        preload_out   <= 1'b0;  
        dividend_out  <= 32'b0;  
        divisor_out   <= 32'b0;  
        remainder     <= 32'b0;  
        quotient      <= 32'b0;  
        valid         <= 1'b0;  
    end  
    else begin  
        start_out    <= start;  
        preload_out   <= preload;  
        dividend_out  <= dividend;  
        divisor_out   <= divisor;  
        remainder     <= remainder_in[63:32];  
        quotient      <= quotient_in;  
        valid         <= valid_in;  
    end  
end  
endmodule
```

圖九、Interface.v

Interface.v 負責外部測試與內部運算間的訊號同步與隔離。電路採時脈正緣觸發、同步低準位重置，確保系統重置時所有輸出端清零。在正常運作階段，Interface.v 將外部輸入的 start、preload、dividend、divisor、remainder_in、quotient_in、valid_in 透過暫存器鎖存一個時鐘週期後同步傳遞至輸出端。此設計有效對齊所有訊號的時序，避免外部延遲或雜訊影響內部電路的穩定性。

(c)Divisor.v

```
module Divisor(  
    input clk, rst_n, preload,  
    input [31:0] divisor,  
    output reg [31:0] divisor_out  
);  
always@(posedge clk)  
begin  
    if (!rst_n) divisor_out<=32'b0;  
    else if (preload) divisor_out<=divisor;  
end  
endmodule
```

圖十、Interface.v

Divisor.v 為一個由 32 位元同步暫存器。正緣觸發 (posedge clk)、具備同步低準位重置 (rst_n) 功能以初始化。運算初期，透過訊號 preload 為高準位來寫入除數；載入後 preload 降為低準位，暫存器保持該數值不變，確保在整個乘法疊代過程中，能持續穩定提供給 ALU 進行運算。

(d)ALU.v

```
module ALU (  
    input [31:0] src_1, src_2,  
    input [1:0] funct_code,  
    output reg ALU_carry,  
    output reg [31:0] ALU_result  
);  
always @(*) begin  
    case(funct_code)  
        2'b01: {ALU_carry, ALU_result}={1'b0,src_1}-{1'b0,src_2};  
        2'b10: {ALU_carry, ALU_result}={1'b0, src_1}+(1'b0, src_2);  
        default: {ALU_carry, ALU_result}={1'b0, src_1};  
    endcase  
end  
endmodule
```

圖十一、ALU.v

ALU.v 負責資料加減法運算，根據輸入的控制碼 funct_code 切換運算模式。ALU_carry 訊號，反映減法是否發生借位情況；而低 32 位元則作為資料結果 ALU_result 輸出。借位訊號將回傳給 Control.v，成為除法器條件寫回架構中的判斷依據。

(e) Remainder.v

```
module Remainder (  
    input clk, rst_n, preload, wr_ctrl, sll_ctrl,  
    input [31:0] dividend,  
    input [31:0] ALU_result,  
    output reg [63:0] remainder_out  
);  
always @(posedge clk) begin  
    if (!rst_n)  
        remainder_out <= 64'b0;  
    else if (preload) remainder_out <= {32'b0, dividend};  
    else if (sll_ctrl) remainder_out <= remainder_out << 1;  
    else if (wr_ctrl)  
        begin  
            remainder_out[63:32] <= ALU_result;  
            remainder_out[0] <= 1'b1;  
        end  
    end  
end  
endmodule
```

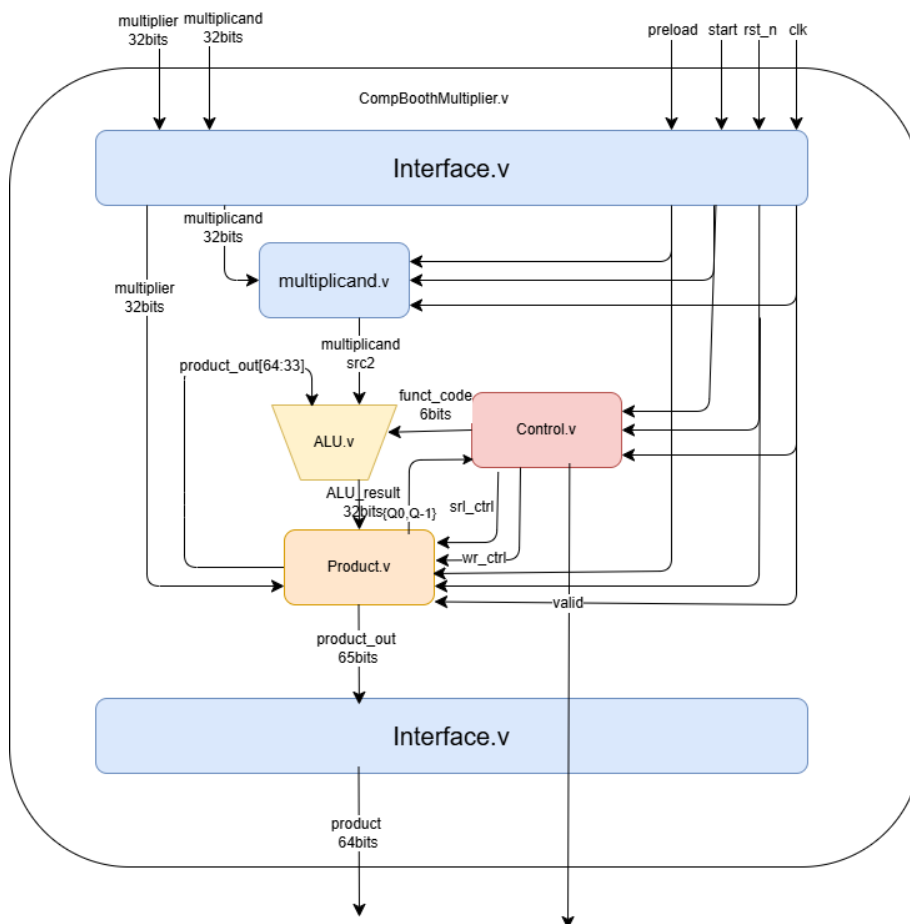
Remainder.v 負責儲存餘數並同步累積商數。在初始化階段 (preload 有效)，將 32 位元被除數載入暫存器的低 32 位元、將高 32 位元清零。當移位訊號 sll_ctrl 觸發時，整體 64 位元資料向左移位，被除數位元逐步推入高位半部供 ALU 進行減法判斷。當控制單元判定夠減，發出寫回訊號 wr_ctrl 時，將 ALU 計算後的新餘數 (ALU_result) 更新至暫存器的高 32 位元。同時，將暫存器的最低位元設為 1'b1，記錄當次商數。

圖十一、ALU.v

(f) Control.v

除法器運算核心的有限狀態機 (FSM)。狀態機劃分為 IDLE、SHIFT、SUB、CHECK 與 DONE 五個階段。IDLE 狀態時 counter 歸零。當接收到 start 訊號後，狀態跳 SHIFT。在 SHIFT 狀態觸發移位訊號 (sll_ctrl)，接著於 SUB 階段指示 ALU 執行減法運算 (funct_code = 2'b01)。在 ALU 運算結果未借位 (即 ALU_MSB == 1'b0，代表夠減) 時，才觸發寫入訊號 (wr_ctrl) 更新暫存器數值。CHECK 階段累加計數器，當疊代滿 32 次後轉入 DONE 狀態並輸出運算完成訊號 (valid)。

Part3. BoothMultiplier



圖十二、BoothMultiplier 架構圖

在 Unsigned Multiplier 基礎邏輯上做些微更動

(a) Interface.v, Multiplicand.v 與 Unsigned Multiplier 相同不需修改

(b) ALU.v

與 Unsigned Multiplier 的 ALU.v 差在多了減法，當 BoothMultiplier 遇到連續的 1 時，用一加一減來取代連續加法，所以 ALU 須根據 Control 傳來的 funct_code，動態切換執行加法或減法。

(c) Product.v

相較於 Unsigned Multiplier 擴充一個隱藏位元並改變移位規則。在原本的第 0 個 bit 的右邊多 1-bit 的暫存空間，初始值設為 0。每次移位時，原本的 LSB 會掉進這裡。向右移位時，最左邊 (MSB) 不只補 0 或補 carry，而是複製自己的符號位元 (Sign Bit) 確保有號數 (負數) 在移位的過程中，正負號不會跑掉。

(d) Control.v

BoothMultiplier 看 MSB 與前一次移出去的位元，這 2 個 bits 的組合：

01：觸發 wr_ctrl，指示 ALU 執行加法

10：觸發 wr_ctrl，指示 ALU 執行減法

00 或 11：不觸發 wr_ctrl，直接進入移位 (不加也不減)

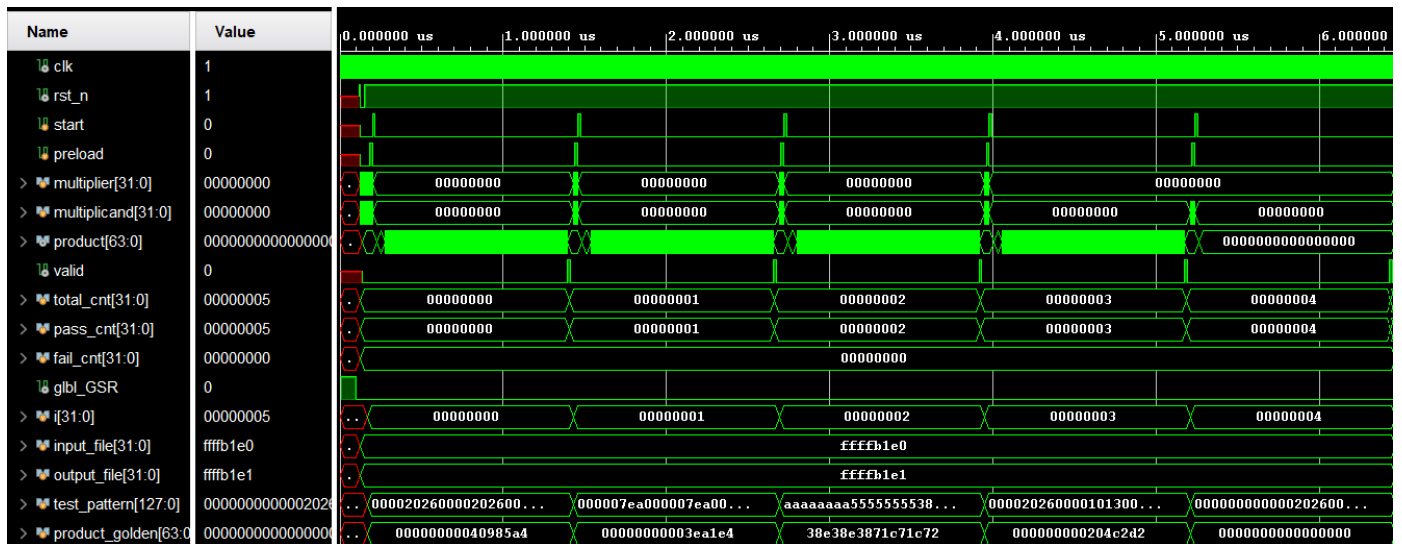
(e) CompMultiplier.v

將 Product 模組裡新增的位元牽出來，連同 MSB 一起送給 Control 做判斷；同時加寬連接 ALU 的 funct_code 線路，讓它能傳遞加/減法指令，除此之外不變。

2. Behavioral Simulation test results

Part1. Multiplier

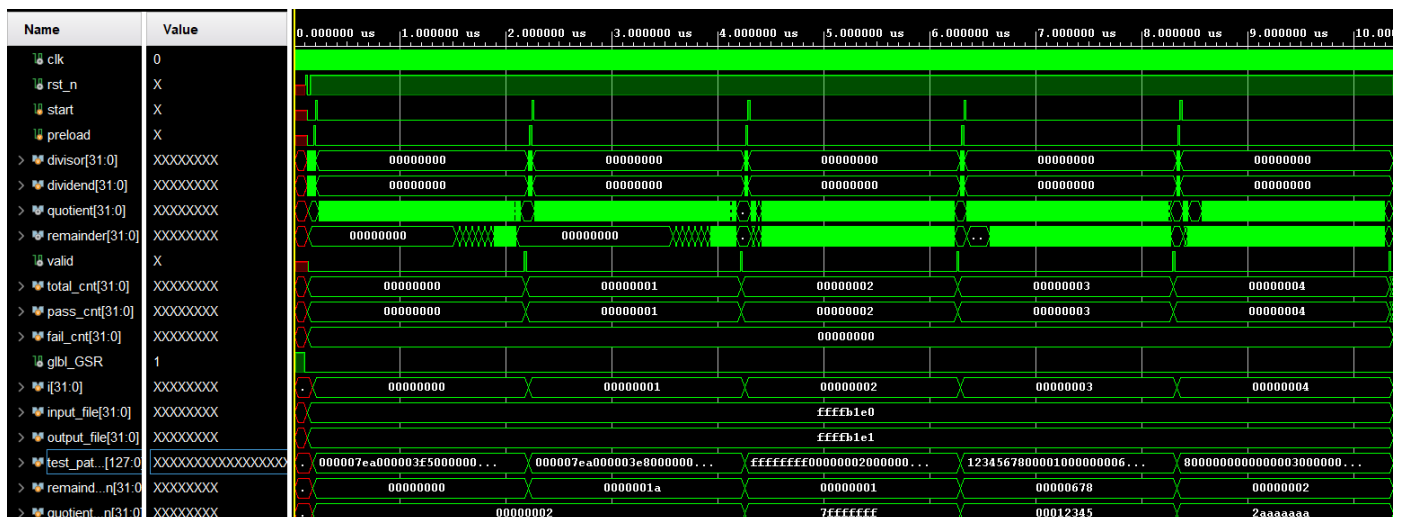
```
# run all
[PASS] 8230 * 8230 = 67732900
[PASS] 2026 * 2026 = 4104676
[PASS] 2863311530 * 1431655765 = 4099276458915470450
[PASS] 8230 * 4115 = 33866450
[PASS] 0 * 8230 = 0
-----
[SUMMARY] Total: 5, Pass: 5, Fail: 0
[RESULT] PASS: All tests passed.
```

- ✓ 控制時序正確：start 觸發運算後，經 32 週期，系統拉高 valid 訊號表示完成。
- ✓ 運算結果無誤：在 valid 觸發瞬間，product 匯流排同步輸出正確的 64 位元乘積結果。
- ✓ 驗證全數通過：Testbench 計數器顯示 [pass]，且 fail 為 0。

Part2. Divider

```
# run all
[PASS] 2026 / 1013 = Q:2 R:0
[PASS] 2026 / 1000 = Q:2 R:26
[PASS] 4294967295 / 2 = Q:2147483647 R:1
[PASS] 305419896 / 4096 = Q:74565 R:1656
[PASS] 2147483648 / 3 = Q:715827882 R:2
-----
[SUMMARY] Total: 5, Pass: 5, Fail: 0
[RESULT] PASS: All tests passed.
```



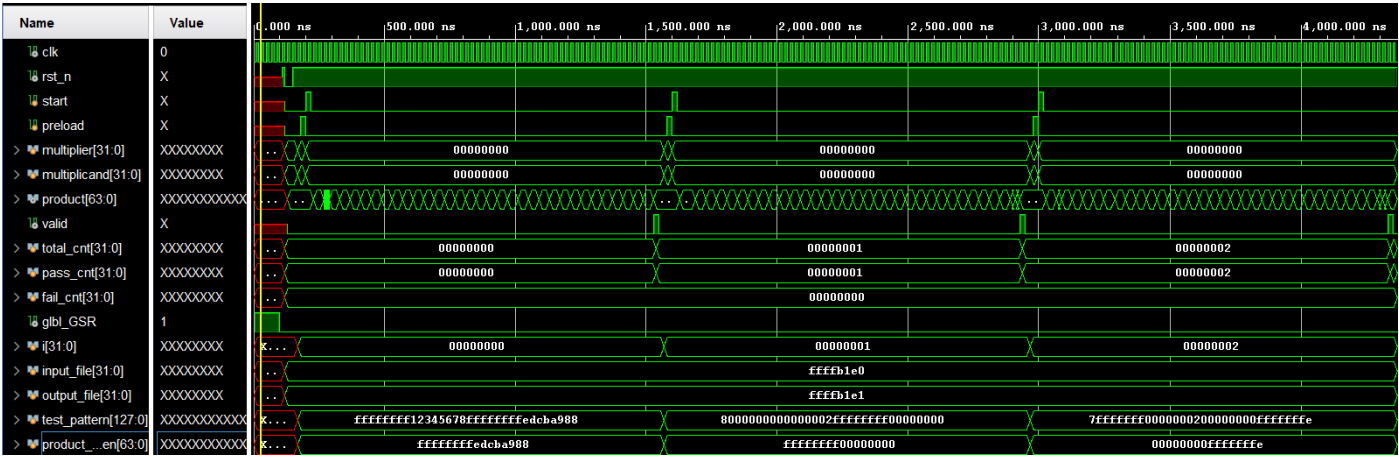
- ✓ 控制時序正確：start 觸發運算後，經 32 週期，系統拉高 valid 訊號表示完成。
- ✓ 運算結果無誤：在 valid 觸發瞬間，product 匯流排同步輸出正確的 64 位元乘積結果。
- ✓ 驗證全數通過：Testbench 計數器顯示 [pass]，且 fail 為 0。

Part3. BoothMultiplier

```
# run all
[PASS] -1 * 305419896 = -305419896
[PASS] -2147483648 * 2 = -4294967296
[PASS] 2147483647 * 2 = 4294967294

-----

[SUMMARY] Total: 3, Pass: 3, Fail: 0
[RESULT] PASS: All tests passed.
```



- ✓ 控制時序正確：start 觸發運算後，系統依序執行疊代，並於完成時拉高 valid 訊號。
- ✓ 有號數運算無誤：包含大量負數（如 ffffffff...）的測資，product 仍能在 valid 觸發瞬間同步輸出正確的 64 位元乘積。
- ✓ 驗證全數通過：Testbench 計數器顯示 [pass]，且 fail 為 0。

3. Synthesis Result
Part1. Multiplier

Name	Slice LUTs (134600)	Slice Registers (269200)	Bonded IOB (400)	BUFGCTRL (32)
CompMultiplier	92	238	133	1
M1 (Interface)	18	131	0	0
M3 (Product)	33	65	0	0

Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 2.491 ns	Worst Hold Slack (WHS): 0.139 ns	Worst Pulse Width Slack (WPWS): 8.500 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 642	Total Number of Endpoints: 642	Total Number of Endpoints: 239

All user specified timing constraints are met.

硬體資源 (Hardware Utilization)：整體面積極小化，消耗 92 個 LUTs 與 238 個暫存器 (Registers)。暫存器精準分配於 Interface 與 Product 模組，成功發揮循序運算架構共用硬體、節省面積的優勢。

時序報告 (Timing Summary)：時序完美收斂，所有約束條件皆達標（Failing Endpoints 為 0）。建立時間寬裕度 (WNS) 達 +2.491 ns，保持時間寬裕度 (WHS) 為 +0.139 ns，物理延遲下無時序違規。

Part2. Divider

Name ^{^1}	Slice LUTs (134600)	Slice Registers (269200)	Bonded IOB (400)	BUFGCTRL (32)
▼ N CompDivider	106	238	133	1
I M1 (Interface)	33	131	0	0
I M3 (Remainder)	36	64	0	0

Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 3.491 ns	Worst Hold Slack (WHS): 0.139 ns	Worst Pulse Width Slack (WPWS): 9.500 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 579	Total Number of Endpoints: 579	Total Number of Endpoints: 239
All user specified timing constraints are met.		

硬體資源 (Hardware Utilization)：整體面積極小化，消耗 106 個 LUTs 與 238 個暫存器。從階層圖可見，暫存器精準分布於 Interface (131 個) 與 Remainder 模組 (64 個)，印證了將商數與餘數合併於同一個 64 位元移位暫存器的硬體資源共用策略。

時序報告 (Timing Summary)：時序完美收斂，所有約束條件皆順利達標（Failing Endpoints 為 0）。建立時間寬裕度 (WNS) 達 +3.491 ns，保持時間寬裕度 (WHS) 為 +0.139 ns，物理延遲下無時序違規。

Part3. BoothMultiplier

Name ^{^1}	Slice LUTs (134600)	Slice Registers (269200)	Bonded IOB (400)	BUFGCTRL (32)
▼ N CompBoothMultiplier	129	238	134	1
I M1 (Interface)	18	131	0	0
I M3 (Product)	70	65	0	0

Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 3.491 ns	Worst Hold Slack (WHS): 0.139 ns	Worst Pulse Width Slack (WPWS): 9.500 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 640	Total Number of Endpoints: 640	Total Number of Endpoints: 239
All user specified timing constraints are met.		

硬體資源 (Hardware Utilization)：整體消耗 129 個 LUTs 與 238 個暫存器。相較於基礎乘法器，LUTs 數量微幅增加，精準反映了 Booth 演算法需擴充 ALU 減法功能及雙位元判斷控制邏輯的硬體開銷。此外，Product 模組使用了 65 個暫存器，呈現 64 位元乘積加上 1 個 Booth 專用參考位元的特殊架構設計。

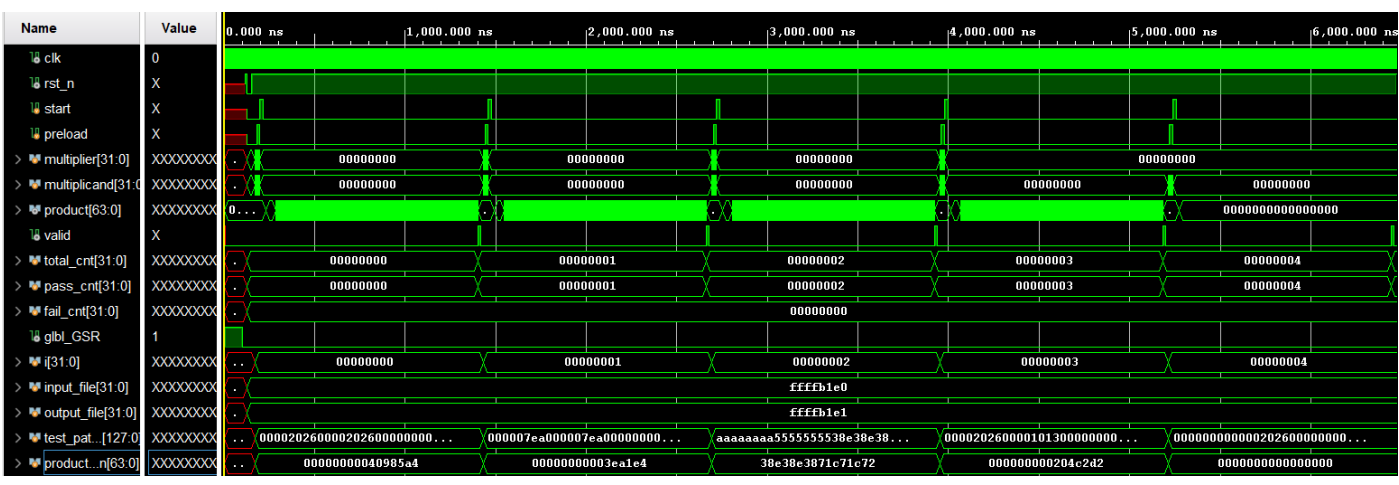
時序報告 (Timing Summary)：時序完美收斂，所有約束條件皆順利達標（Failing Endpoints 為 0）。建立時間寬裕度 (WNS) 達 +3.491 ns，保持時間寬裕度 (WHS) 為 +0.139 ns，物理延遲下無時序違規。

4. Post-Synthesis Timing Simulation

Part1. Multiplier

```
# run all
[PASS] 8230 * 8230 = 67732900
[PASS] 2026 * 2026 = 4104676
[PASS] 2863311530 * 1431655765 = 4099276458915470450
[PASS] 8230 * 4115 = 33866450
[PASS] 0 * 8230 = 0

-----
[SUMMARY] Total: 5, Pass: 5, Fail: 0
[RESULT] PASS: All tests passed.
```



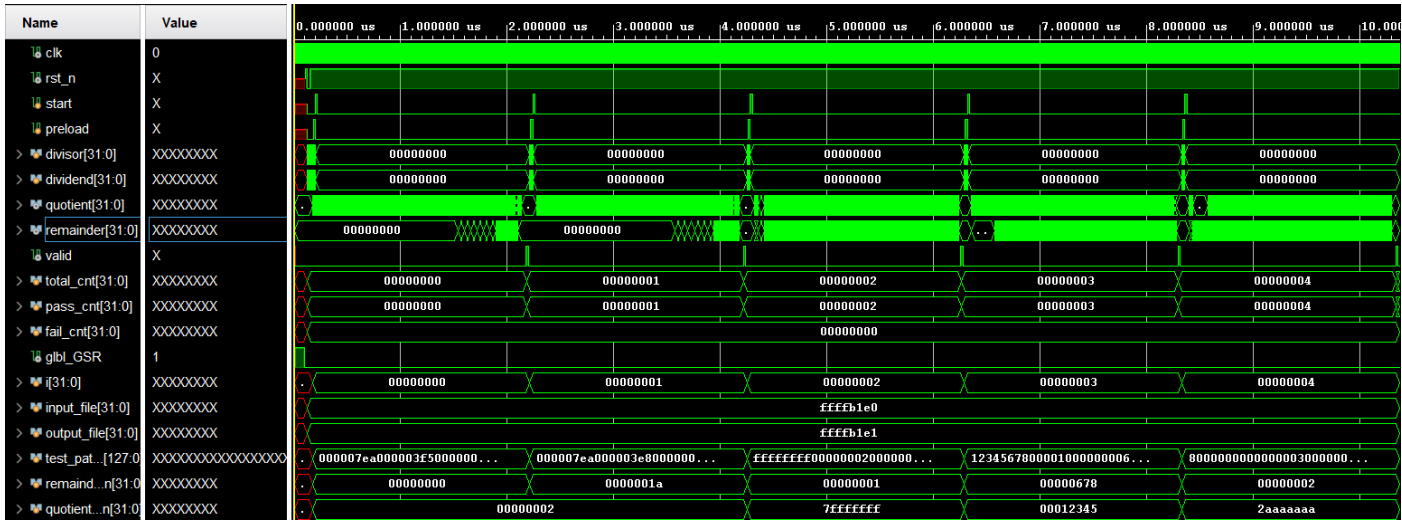
邏輯正確性驗證 (TCL Console)：如終端機截圖所示，5 筆測資皆全數獲得 [PASS]。這證明 Verilog 程式碼經過合成轉換為真實邏輯閘（Gate-level Netlist）後，運算功能依正確。

真實時序穩定度 (Waveform)：從波形圖可觀察到，導入真實的邏輯閘與繞線延遲（Gate & Routing Delay）後，整體電路未因時序偏移而產生誤動作。當 valid 訊號觸發時，product 匯流排仍能穩定輸出正確的乘積結果。

Part2. Divider

```
[PASS] 2026 / 1013 = Q:2 R:0
[PASS] 2026 / 1000 = Q:2 R:26
[PASS] 4294967295 / 2 = Q:2147483647 R:1
[PASS] 305419896 / 4096 = Q:74565 R:1656
[PASS] 2147483648 / 3 = Q:715827882 R:2

-----
[SUMMARY] Total: 5, Pass: 5, Fail: 0
[RESULT] PASS: All tests passed.
```

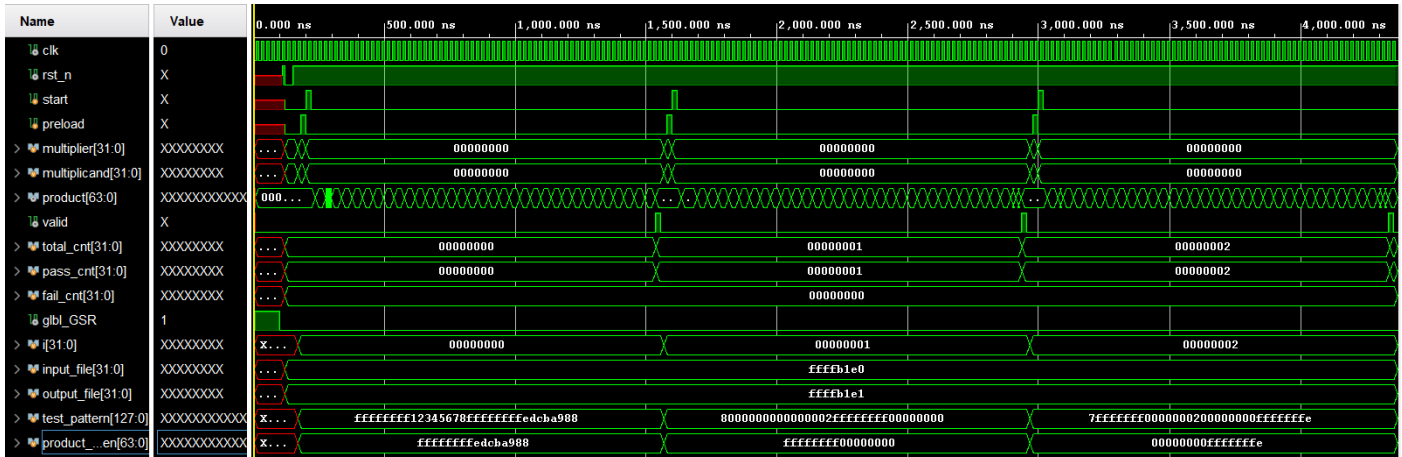


邏輯正確性驗證：如終端機截圖所示，5 筆測資皆全數獲得 [PASS]。這證明 Verilog 程式碼經過合成轉換為真實邏輯閘（Gate-level Netlist）後，運算功能依正確。

真實時序穩定度 (Waveform)：在導入真實的邏輯閘與繞線延遲（Gate & Routing Delay）後，整體狀態機仍正確運作。當 valid 訊號觸發時，quotient (商數) 與 remainder (餘數) 雙匯流排皆能同步且輸出正確的十六進位結果。

Part3. BoothMultiplier

```
# run all
[PASS] -1 * 305419896 = -305419896
[PASS] -2147483648 * 2 = -4294967296
[PASS] 2147483647 * 2 = 4294967294
-----
[SUMMARY] Total: 3, Pass: 3, Fail: 0
[RESULT] PASS: All tests passed.
```



邏輯正確性驗證 (TCL Console)：如終端機截圖所示，3 筆測資皆全數獲得 [PASS]。這證明 Verilog 程式碼經過合成轉換為真實邏輯閘（Gate-level Netlist）後，運算功能依正確。

真實時序穩定度 (Waveform)：從波形圖可觀察到，導入真實的邏輯閘與繞線延遲（Gate & Routing Delay）後，整體電路未因時序偏移而產生誤動作。當 valid 訊號觸發時，product 匯流排仍能穩定輸出正確的乘積結果。

5. Conclusion (結論與心得)

在此次「模組化循序乘除法器與 Booth 乘法器」的專題實作中，由於先前參與過今年的 IC Contest，對於獨立撰寫乘除法器的 RTL code 已具備一定的熟悉度，因此在核心邏輯的實現上並未感到太困難。然而，這次實作帶來最大的挑戰與收穫，反而是「必須嚴格遵循助教定義的 Datapath 架構圖」來進行設計。

過去在寫 code 時較習慣以功能實現為導向，但這次被要求將電路精細拆解，甚至需加入如 Interface.v 這種初看似乎可以省略的 I/O 隔離模組。這段過程讓我深刻體會到，在規範嚴謹的專案中，模組化的邊界定義與訊號隔離對於系統穩定性有多麼重要。同時，我也將課堂上的理論具體落實，包含利用單一 64 位元暫存器同時存放餘數與商數以達到極致的資源共用，以及成功將基礎乘法器升級為完美支援有號數運算的 Booth Multiplier 架構。

此外，本次專題最令我感到新奇的環節是後續的合成流程。這套流程與我在實驗室所熟悉的 Design Compiler (ASIC flow) 有著相當不同的觀念與操作體驗。透過這次專題，我不僅跨域體驗了不同的 EDA Toolchain，也對電路從行為級描述到物理層級的轉換有了更全面的理解。這些融合了實戰比賽、課堂理論與不同工具鏈的歷練，相信都將為我未來在開發 AI 加速器底層硬體時，提供更深厚且全面的架構思維。